

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 钱宸玮 学号 2024K8009929010 专业 计算机科学与技术
实验项目编号 2 实验名称 简单功能处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- **设计**步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

1 单周期 MIPS/RISC-V 处理器设计

本次实验使用了 32 位 MIPS 指令集。首先参照课程资料中的译码表, 指令类型分析对本次实验涉及到的 45 条指令有了大概的理解, 并完成了自己的指令译码表。

设计中, 主要考虑如图的单周期 CPU 运行的五个阶段:

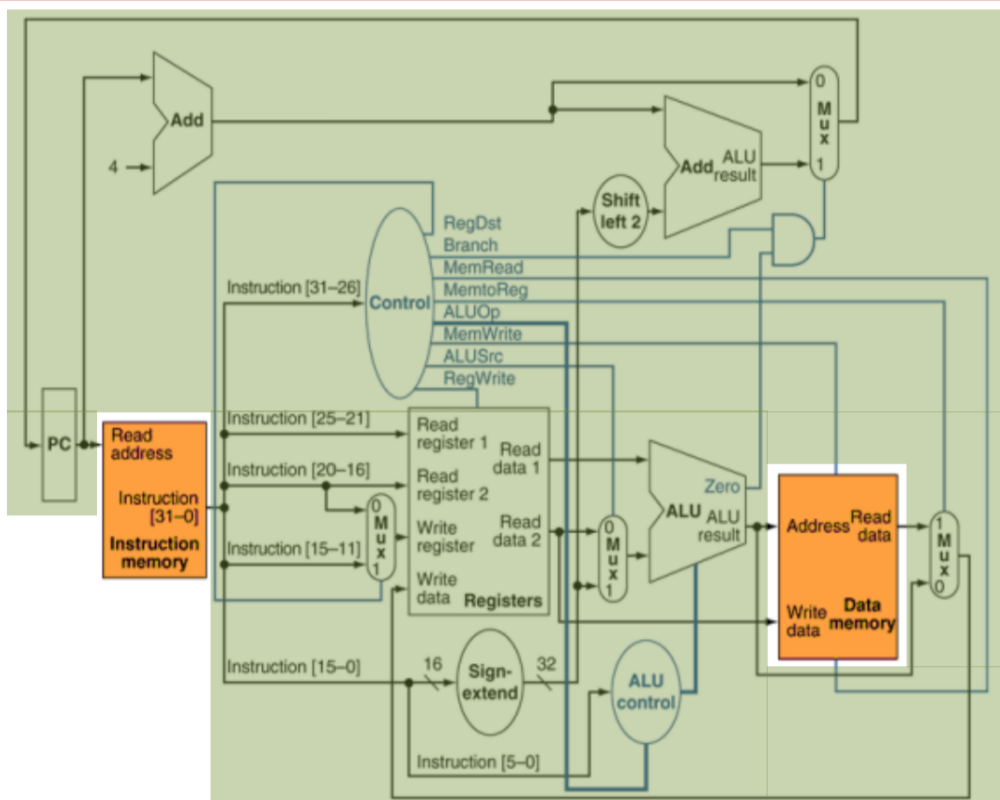


图 1: 单周期处理器结构图

1.1 取指

完成取值操作的主要部分是 PC(程序计数器)。

PC 储存下一条指令的地址。无跳转/分支指令时, PC 每时钟周期 +1, 考虑到地址对齐实际为 +4. 遇到分支, 跳转指令时, 根据具体指令内容更新为目标地址。

1.2 译码

译码操作主要体现于控制单元, 寄存器堆和对指令内容的处理。

控制单元将指令拆分为若干字段, 根据指令类型的不同对每个字段的处理也不同; 根据指令的高 6 位 (opcode) 控制寄存器堆, ALU, PC 等部件的行为。

寄存器堆根据指令内容从指定端口读出或写入数据, 数据的来源/目标可以是立即数, 内存等。

根据指令类型, 对指令中的立即数做扩展处理。

1.3 执行

执行操作主要包含 ALU, 移位器等。

ALU 接受 ALU 控制单元给出的 ALUOp 及来自寄存器堆/立即数的操作数, 并计算出结果。同时, ALU 也计算内存读写指令的目标地址。

移位器进行左移, 算术右移和逻辑右移操作。

1.4 访存

访存指令向 ALU 计算出的内存地址写入/读出指定内容, 写入数据来自寄存器堆, 读出数据写入寄存器堆。内存读写由内存读写信号控制, 不同的指令可以实现对内存单元中数据不同字节的操作。

1.5 写回

写回阶段, 根据 MemtoReg 信号决定写回寄存器堆的数据来自 ALU 还是内存读出, 并确定写入寄存器堆的地址。

2 多周期 MIPS/RISC-V 处理器设计(如有)

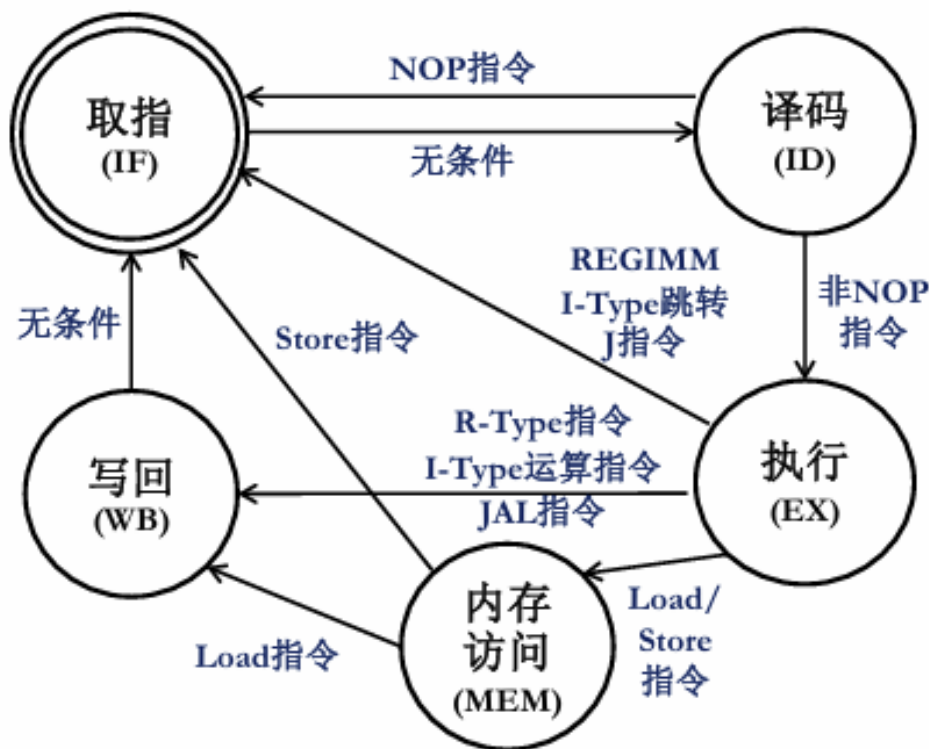


图 2: 多周期处理器状态机

多周期处理器的五个阶段与单周期处理器一致, 区别在于多周期处理器在每个时钟周期完成一个阶段, 单周期处理器的所有阶段在一个时钟周期内完成。

单周期处理器的控制信号完全由指令类型决定; 多周期处理器的控制信号由当前阶段和指令类型共同决定。

多周期处理器设计的主要部分是三段式状态机; 第一段状态机实现状态复位与状态更新, 第二段状态机规定每个状态的下个状态, 第三段状态机规定每种指令在每个状态中的控制信号。其中第一段状态机使用时序逻辑, 第二段和第三段状态机使用时序逻辑。

多周期处理器使用若干中间寄存器在不同阶段间传递数据。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 单周期 MIPS/RISC-V 处理器设计

2.1 取指

PC 的设计如下:

```

always @(posedge clk) begin //程序计数器
    if (rst == 1'b1) begin
        PC_reg <= 32'b0;
    end
    else begin
        if (branch == 1'b1) begin //分支处理
            PC_reg <= PC_reg + 4 + se_target_offset;
        end
        else if (jump == 1'b1) begin //跳转处理
            PC_reg <= jump_address;
        end
        else begin
            PC_reg <= PC_reg + 4;
        end
    end
end
assign PC = PC_reg;

```

图 3: 程序计数器代码

复位信号同步,跳转,分支指令单独处理,若无跳转或分支,则 $PC + 4$ 。

PC 的声明为 wire PC, 无法直接在时序逻辑 always 块中赋值, 故使用 reg 声明的临时变量 PC_reg 在 always 块中代替 PC。

2.2 译码

寄存器堆的设计已在实验 1 中完成,此处不再给出代码。

涉及到寄存器堆的使能,输入输出信号处理如下:

assign	RF_raddr1	=	isRtype isREGIMM isRtypeb isRtypec isRtypep isRtypes	? rs ? rs ? rs ? rs ? base ? base	: : : : : :	//R-type //REGIMM //R-type分支 //R-type计算 //R-type内存读地址 //R-type内存写
assign	RF_raddr2	=	isRtype isREGIMM islwlr isRtypeb isRtypes	? rt ? 5'b00000 ? rt ? rt ? rt	: : : : :	//R-type //REGIMM //lw/lwr //R-type分支 //R-type内存写数据
assign	RF_wen	=	(movn == 1) (movz == 1) isRtype && (func[5] == 1'b1 func[5:3] == 3'b000) isjalr islui isRtypec isRtypep isRtypes	? 1'b1 ? 1'b1 ? 1'b1 ? 1'b1 ? 1'b1 ? 1'b0	: : : : : :	//数据移动指令 //R-type运算/位移指令 //R-type jal/jalr指令 //lui //R-type计算 //R-type内存读
assign	RF_waddr	=	(movn == 1) (movz == 1) isRtype isjalr islui isRtypec isRtypep isRtypes	? rd ? rd ? 5'b11111 ? rt ? rt ? rt	: : : : : :	//数据移动指令 //R-type //R-type jal/jalr指令 //lui //R-type计算 //R-type内存读
assign	RF_wdata	=	isjalr isRtypep (movn == 1) (movz == 1) islui (isRtypec (isRtype && func[5:3] != 3'b00)) isRtype && func[5:3] == 3'b0	? PC + 8 ? mem_data ? RF_rdata1 ? im_immediate ? alu_result ? shifter_result 32'b0	: : : : : : :	//jal或jalr写入PC + 8 //R-type内存读 //R-type指令处理 //lui单独处理 //计算类非位移指令, alu计算结果作为写入数据 //计算类位移指令

图 4: 寄存器堆相关信号

寄存器堆的写使能,写入地址,写入数据,读出地址取决于当前指令的类型及具体内容。

控制单元对指令的拆分,判断如下:

```

//指令相关
wire [5:0]      opcode;      //指令31-26位
wire [4:0]      rs;          //指令25-21位
wire [4:0]      rt;          //指令20-16位
wire [4:0]      rd;          //指令15-11位
wire [4:0]      shmat;       //指令10-6位
wire [5:0]      func;        //指令5-0位

wire [15:0]      immediate;   //I-type指令15-0位

wire [26:0]      instr_index; //J-type指令26-0位

wire [31:0]      se_immediate; //符号位扩展后的立即数
wire [31:0]      zo_immediate; //零扩展后的立即数
wire [31:0]      lm_immediate; //左移16位后的立即数

wire [15:0]      offset;      //I-type分支/内存读/内存写/REGIMM指令15-0位
wire [17:0]      target_offset; //I-type分支offset左移2位
wire [31:0]      se_target_offset; //I-type分支扩展后的offset
wire [31:0]      se_offset;    //I-type内存读写offset部分符号位扩展

wire [4:0]      base;          //I-type内存读写指令25-21位

```

图 5: 指令处理

```

assign isRtype      = (opcode == 6'b000000);
assign isREGIMM     = (opcode == 6'b000001);
assign isJtype      = (opcode[5:1] == 5'b00001);
assign isItypeb     = (opcode[5:2] == 4'b0001);
assign isItypec     = (opcode[5:3] == 3'b001);
assign isItype1     = (opcode[5] & (~opcode[3]));
assign isItypes     = (opcode[5] & opcode[3]);
assign islui        = (opcode == 6'b001111);
assign isjalr       = ((isRtype && func == 6'b001001) || opcode == 6'b000011);
assign isItypec1    = (opcode[5:2] == 4'b0010);
assign isItypec1    = (opcode[5:2] == 4'b0011);
assign islwlr       = (opcode == 6'b100010 || opcode == 6'b100110);
assign isj          = (opcode == 6'b000010 || opcode == 6'b000011);

```

图 6: 指令处理

根据指令的高 6 位(opcode)可以确定指令的类型,如 R-type,I-type 等。根据该类型可以确定应该如何将指令拆分为不同字段。

部分指令包含非 32 位立即数,需要对这些立即数进行扩展处理,如下图:

```

assign se_immediate = {{16{immediate[15]}},immediate}; //符号位扩展
assign zo_immediate = {{16{1'b0}},immediate}; //零扩展
assign lm_immediate = {immediate,{16{1'b0}}}; //左移16位

assign offset       = Instruction[15:0];
assign target_offset = {offset,{2{1'b0}}}; //左移2位
assign se_target_offset = {{14{target_offset[17]}},target_offset}; //符号位扩展
assign se_offset     = {{16{offset[15]}},offset}; //符号位扩展

```

图 7: 立即数扩展

2.3 执行

本实验的 ALU 添加了新的功能,包含无符号数比较,逻辑异或/同或。修改部分如下:

```

assign AND = A & B; //按位与, 000
assign OR = A | B; //按位或, 001
assign SLT = (Overflow == 0) ? (ADD_SUB[31] == 1 ? 32'b0 : 32'b1) : (ADD_SUB[31] == 1 ? 32'b0 : 32'b1); //比较在符号数, 通过观察A-B的符号位实现, 同时考虑Overflow是否存在, 若存在则取反, 111
assign XOR = A ^ B; //按位异或, 100
assign NOR = ~ A ^ B; //按位同或, 101
assign SLTU = (tempcarryout == 1'b0) ? 32'b1 : 32'b0; //无符号数比较, 011

assign Result =
  (ALUop == 3'b000) ? AND :
  (ALUop == 3'b001) ? OR :
  (ALUop == 3'b010 || ALUop == 3'b110) ? ADD_SUB :
  (ALUop == 3'b100) ? XOR :
  (ALUop == 3'b101) ? NOR :
  (ALUop == 3'b111) ? SLT :
  (ALUop == 3'b011) ? SLTU :
  32'b0;
//用多层多目运算符实现多路选择器

assign Overflow =
  (ALUop[2] == 0) ? (((~A[31] && ~B[31]) && ADD_SUB[31]) || ((A[31] && B[31]) && ~ADD_SUB[31])) ? 1'b1 : 1'b0 :
  (ALUop[2] == 1) ? (((~A[31] && B[31]) && ADD_SUB[31]) || ((A[31] && ~B[31]) && ~ADD_SUB[31])) ? 1'b1 : 1'b0 :
  1'b0;
//正数 + 正数 == 负数/负数 + 负数 == 正数/正数 - 负数 == 负数/负数 - 正数 == 正数, 则有符号数加法溢出

assign (tempcarryout, ADD_SUB) = A + realB + cin;
//拼接一个完整结果, 可通过tempcarryout得出是否存在 A + B < A || A + B < 0 || A - B > A, 即无符号数加法溢出(进位); ADD为010, SUB为110

assign Carryout =
  (ALUop[2] == 1'b0) ? tempcarryout :
  (ALUop[2] == 1'b1) ? ~tempcarryout :
  1'b0; //减法时取反

assign Zero = (Result == 32'b0); //Result是否为0

endmodule

```

图 8: 修改后的 ALU

其中同或和异或功能可直接通过 Verilog 语法实现; 无符号数比较通过减法结果的进位实现。
ALU 控制单元实现如下:

```

assign alu_A = RF_rdata1;
assign alu_B = isItypecnl ? se_immediate : //I-type非逻辑类运算指令, 与符号位扩展立即数运算
  isItypecl ? zo_immediate : //I-type逻辑运算指令, 与零扩展立即数运算
  isItypepl || isItypes ? se_offset : //I-type内存读写指令
  RF_rdata2; //其他计算, 取寄存器二号读出端口

assign alu_op = isItypepl || isItypes ? 3'b010 : //内存读写地址计算
  isRtype & (func[3:2] == 2'b00) ? {func[1], 2'b10} : //add/sub
  isRtype & (func[3:2] == 2'b01) ? {func[1], 1'b0, func[0]} : //逻辑运算
  isRtype & (func[3:2] == 2'b10) ? {~func[0], 2'b11} : //比较运算
  opcode[2:1] == 2'b00 ? {opcode[1], 2'b10} : //立即数加法
  opcode[2:1] == 1'b1 ? {opcode[1], 1'b0, opcode[0]} : //立即数逻辑运算
  opcode[2:1] == 2'b01 ? {~opcode[0], 2'b11} : //立即数比较
  3'b000;

```

图 9: ALU 控制单元

ALU 的第一个操作数始终来自于寄存器堆的一号读出端口; 第二个操作数根据指令内容可以来自寄存器堆或立即数。ALUop 来自指令本身。

移位器的实现如下:

```

assign leftmove = A << B; //将A左移B位, 低位补0
assign crightmove = $signed(A) >>> B; //将有符号数A算数右移B位, 高位补1
assign lrightmove = A >> B; //将A逻辑右移B位, 高位补0

assign Result = Shiftop == 2'b00 ? leftmove :
  Shiftop == 2'b11 ? crightmove :
  Shiftop == 2'b10 ? lrightmove :
  32'b0;

```

图 10: 移位器

其中左移功能, 逻辑右移功能直接通过 Verilog 语法实现。算术右移功能通过将操作数强制类型转换为有符号数, 使用 Verilog 语法实现。

2.4 访存

访存指令的实现如下; 此处以 load 指令为例, store 指令的实现基本一致。

```

assign      temp_address  =      (isItype1 || isItypes) ?      alu_result : //读写地址均为立即数+寄存器base
                                                    32'b0;

assign      Address       =      {temp_address[31:2], 2'b0}; //地址对齐

assign      MemRead       =      isItype1;

assign      MemWrite      =      isItypes;

assign      temp_read_strb =      opcode[2:0] == 3'b011 ?      4'b1111 : //lw指令, 读取完整数据
                                opcode[1:0] == 2'b01 & (temp_address[1:0] == 2'b00) ?      4'b0011 : //lh指令, 读取两个字节, 低两位
                                opcode[1:0] == 2'b01 & (temp_address[1:0] == 2'b10) ?      4'b1100 : //高两位
                                opcode[1:0] == 2'b00 & (temp_address[1:0] == 2'b00) ?      4'b0001 : //lb指令, 读取一个字节, 低8位
                                opcode[1:0] == 2'b00 & (temp_address[1:0] == 2'b01) ?      4'b0010 : //次低8位
                                opcode[1:0] == 2'b00 & (temp_address[1:0] == 2'b10) ?      4'b0100 : //次高8位
                                opcode[1:0] == 2'b00 & (temp_address[1:0] == 2'b11) ?      4'b1000 : //高8位
                                4'b0000;

assign      read_strb     =      (opcode[2:0] == 3'b010) ? ( // LWR 小端序
                                (temp_address[1:0] == 2'b00) ? 4'b0001 : // 偏移0: 仅读byte 0
                                (temp_address[1:0] == 2'b01) ? 4'b0011 : // 偏移1: 读byte 1, 0
                                (temp_address[1:0] == 2'b10) ? 4'b0111 : // 偏移2: 读byte 2, 1, 0
                                4'b1111 // 偏移3: 读byte 3, 2, 1, 0
                                ) :
                                (opcode[2:0] == 3'b110) ? ( // LWR 小端序
                                (temp_address[1:0] == 2'b00) ? 4'b1111 : // 偏移0: 读byte 3, 2, 1, 0
                                (temp_address[1:0] == 2'b01) ? 4'b1110 : // 偏移1: 读byte 3, 2, 1
                                (temp_address[1:0] == 2'b10) ? 4'b1100 : // 偏移2: 读byte 3, 2
                                4'b1000 // 偏移3: 仅读byte 3
                                ) :
                                temp_read_strb;

```

图 11: 访存指令实现

```

assign      mem_data      =      (opcode[2:0] == 3'b010) ? ( // LWR 小端序拼接
                                (temp_address[1:0] == 2'b00) ? {Read_data[7:0], RF_rdata2[23:0]} : // 把总线最低8位移到寄存器最高8位
                                (temp_address[1:0] == 2'b01) ? {Read_data[15:0], RF_rdata2[15:0]} : // 把总线低16位移到寄存器高16位
                                (temp_address[1:0] == 2'b10) ? {Read_data[23:0], RF_rdata2[7:0]} : // 把总线低24位移到寄存器高24位
                                Read_data // 满字直接存入
                                ) :
                                (opcode[2:0] == 3'b110) ? ( // LWR 小端序拼接
                                Read_data // 满字直接存入
                                {RF_rdata2[31:24], Read_data[31:8]} : // 把总线高24位移到寄存器低24位
                                {RF_rdata2[31:16], Read_data[31:16]} : // 把总线高16位移到寄存器低16位
                                {RF_rdata2[31:8], Read_data[31:24]} // 把总线最高8位移到寄存器最低8位
                                ) :
                                // 常规指令逻辑
                                Read_data
                                read_strb == 4'b1111 ? {16{Read_data[15]}} : // 1h
                                read_strb == 4'b0011 & opcode[2:0] == 3'b01 ? {{16{1'b0}}, Read_data[15:0]} : // 1hu
                                read_strb == 4'b1100 & opcode[2:0] == 3'b01 ? {{16{Read_data[31]}}}, Read_data[31:16]} : // 1h (高半字)
                                read_strb == 4'b1100 & opcode[2:0] == 3'b01 ? {{16{1'b0}}, Read_data[31:16]} : // 1hu (高半字)
                                read_strb == 4'b0001 & opcode[2:0] == 3'b00 ? {{24{Read_data[7]}}}, Read_data[7:0]} : // 1b
                                read_strb == 4'b0001 & opcode[2:0] == 3'b00 ? {{24{1'b0}}, Read_data[7:0]} : // 1bu
                                read_strb == 4'b0010 & opcode[2:0] == 3'b00 ? {{24{Read_data[15]}}}, Read_data[15:8]} :
                                read_strb == 4'b0010 & opcode[2:0] == 3'b00 ? {{24{1'b0}}, Read_data[15:8]} :
                                read_strb == 4'b0100 & opcode[2:0] == 3'b00 ? {{24{Read_data[23]}}}, Read_data[23:16]} :
                                read_strb == 4'b0100 & opcode[2:0] == 3'b00 ? {{24{1'b0}}, Read_data[23:16]} :
                                read_strb == 4'b1000 & opcode[2:0] == 3'b00 ? {{24{Read_data[31]}}}, Read_data[31:24]} :
                                read_strb == 4'b1000 & opcode[2:0] == 3'b00 ? {{24{1'b0}}, Read_data[31:24]} :
                                32'b0;

```

图 12: 访存指令实现

内存读写的使能信号直接根据当前指令是否是 load/store 类型确定。

访存地址由 ALU 计算,需考虑内存地址对齐。

lw, lh, lb 指令直接通过指令内容确定读取内存数据的哪些部分。

lwl, lwr 指令则根据未对齐的计算出的指令的低两位确定偏移量,根据偏移量读取相应的内存字节。

2.5 写回

该部分的代码已在 2.2 部分一并给出。

2 多周期 MIPS/RISC-V 处理器设计(如有)

2.1 第一段状态机

第一段状态机如图所示：

```

always @(posedge clk) begin                                // 第一段状态机
    if (rst == 1'b1) begin
        current_state <= IF;
    end
    else begin
        current_state <= next_state;
    end
end
end

```

图 13: 第一段状态机

主要实现状态更新和复位。

2.2 第二段状态机

第二段状态机如图所示：

```

always @(*) begin                                          // 第二段状态机
    case (current_state)
        IF: begin
            next_state = ID;                                // 取指完成进入译码
        end
        ID: begin
            next_state = EX;                                // 译码完成进入执行
        end
        EX: begin
            if(isj || isREGIMM || isItypeb || isjr) begin
                next_state = IF;                            // 分支指令, j 指令, jr 指令进入取指阶段
            end
            else if(isItype1 || isItypes) begin
                next_state = MEM;                            // 访存指令进入访存阶段
            end
            else begin
                next_state = WB;                            // 其他指令写回
            end
        end
        MEM: begin
            if(isItype1) begin
                next_state = WB;                            // load 指令访存后写回
            end
            else begin
                next_state = IF;                            // store 指令访存后取指
            end
        end
        WB: begin
            next_state = IF;                                // 写回完成回到取指
        end
        default: begin
            next_state = IF;
        end
    endcase
end
end

```

图 14: 第二阶段状态机

规定了每种指令在每个状态下的的后续状态。

2.3 第三段状态机

第三段状态机如图所示：

```

EX: begin
    if (isItype1 || isItypes) begin          // load/store 指令
        ALUSrcA      = 1'b1;                // ALU 第一个操作数选寄存器rs
        ALUSrcB      = 2'b10;              // ALU 第二个操作数选符号扩展后的立即数
        topALUOp      = 2'b00;              // 执行加法
    end
    else if (islui) begin                    //lui指令
        ALUSrcA      = 1'b1;
        ALUSrcB      = 2'b10;
        topALUOp      = 2'b00;
        UseLui        = 1'b1;
    end
    else if (isItypeb) begin                // 分支指令
        ALUSrcA      = 1'b1;                // 选rs
        ALUSrcB      = 2'b00;                // 选rt
        topALUOp      = 2'b01;              // 执行减法
        PCSource      = 2'b01;              // 若跳转，地址选自ALUOut
        PCWriteCond    = 1'b1;              // 开启条件写使能，由ALU的Zero信号决定是否写PC
    end
    else if (isj) begin                     //j指令
        PCWrite       = 1'b1;                // 开启PC写使能
        PCSource      = 2'b10;              // PC接受跳转地址
    end
    else if (isjr) begin                    //jr指令
        PCSource      = 2'b11;              // PC接受A
        PCWrite       = 1'b1;                // 立即更新PC
    end
    else if (isjal) begin                   //jal指令
        PCWrite       = 1'b1;
        PCSource      = 2'b10;              // 接受跳转地址
    end
    else if (isjalr) begin                  //jalr指令
        PCSource      = 2'b11;              // PC接受A
        PCWrite       = 1'b1;
    end
    else if (isRtype) begin                 // R-type指令
        ALUSrcA      = 1'b1;                // 选rs
        ALUSrcB      = 2'b00;                // 选寄存器rt
        topALUOp      = 2'b10;              // 根据func字段生成的具体操作码
    end
    else if (isItypec) begin                //I-type 计算类指令
        ALUSrcA      = 1'b1;
        ALUSrcB      = 2'b10;
        topALUOp      = 2'b10;
    end
    else if (isREGIMM) begin                // bltz/bgez
        ALUSrcA      = 1'b1;
        ALUSrcB      = 2'b00;
        topALUOp      = 2'b01;
        PCSource      = 2'b01;
        PCWriteCond    = 1'b1;
    end
end

```

图 15: 第三段状态机

图示为执行阶段,根据指令类型决定 ALU 操作数,控制信号,PC 数据来源等控制信号。

2.4 中间寄存器

中间寄存器的实现如图所示：

```

wire    A_B_write;
assign  A_B_write    =      (current_state == ID);
wire    ALUOut_write;
assign  ALUOut_write =      (current_state == ID) || (current_state == EX);
wire    MDR_write;
assign  MDR_write    =      (current_state == MEM) && MemRead_reg;

always @(posedge clk) begin
    if (rst) begin
        IR <= 32'b0;
        MDR <= 32'b0;
        A <= 32'b0;
        B <= 32'b0;
        ALUOut <= 32'b0;
        PC4_temp <= 32'b0;
    end
    else begin
        if (IRWrite) begin
            IR <= Instruction;
            PC4_temp <= alu_result;
        end
        if (MDR_write) begin
            MDR <= Read_data;
        end
        if (A_B_write) begin
            A <= RF_rdata1;
            B <= RF_rdata2;
        end
        if (ALUOut_write) begin
            ALUOut <= alu_result;
        end
    end
end
end

```

图 16: 中间寄存器

数据写入寄存器后根据所处阶段可能进行锁存,防止无意义数据写入寄存器被之后的阶段读取。

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

比如 RTL 代码中出现的逻辑 bug、逻辑仿真和 FPGA 调试过程中的难点等。

出错模块指设计章节中列出的各个模块。

问题类别包括逻辑电路设计、RTL 语法、行为仿真和逻辑综合。

产生原因包括 xxx 概念理解不清、xxx 模块设计环节出现 xxx 问题、xxx 设计不符合 xxx 设计规范等。

• 单周期 MIPS/RISC-V 处理器问题补充。

运行 benchmark 时,发现了较多多路选择器优先级的的问题。应该将判断条件写得更精细,或者将限制条件多的指令放在优先级更高处。

• 多周期 MIPS/RISC-V 处理器问题补充(如有)。请编写你的实验报告内容。

设计中对中间寄存器堆进行了详细锁存,但是除了指令寄存器,其他寄存器的锁存的意义存疑。

对于 Lui,shift,移位类型的指令需要特殊处理,这些情况在早期设计时均未考虑到。

序号	问题类别	出错模块	问题现象	产生原因
1	RTL 语法	各处	编译失败	Verilog 必须先声明后使用
2	RTL 语法	各处	编译失败	拼接时,不需要在某变量两侧加上
3	逻辑电路设计	取指	PC 值不正确	执行分支指令,PC 也要加 4
4	逻辑电路设计	执行	ALUresult 无变化	ALUop 译码时没有考虑到 R-type 的 opcode
5	逻辑电路设计	执行	执行 Lui 指令时结果出错	与 Lui 有共性的其他指令被优先匹配
6	逻辑电路设计	访存	执行访存指令时地址出错	没有考虑到地址对齐
7	逻辑电路设计	访存	执行 Lwl,Lwr,Swr,Swl 时数据出错	对小端序理解不正确

表 1: 实验问题总结

很多代码因为仅参考了理论课提供的资料,出现遗漏的情况较多。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

请总结自己在实验过程中的亮点和不足并补充在此处,针对亮点有什么后续补充计划,针对不足有什么后续提升计划。

可以在表格外补充文字说明。

类别	名称	体会
RTL 语法	Verilog 语法	处理此类问题非常耗费时间,经常完全不知道为什么出错
逻辑电路设计	设计准备	本次实验中有较多需要提前思考的内容,如 PC 的更新,译码表,内存地址对齐,小端序等

表 2: 收获总结

多周期处理器的完成应该快于单周期处理器,但是实际上我在多周期处理器的设计上也花费了相当长的时间。完成试验后再回顾实验过程,我发现自己对多周期处理器的理论课知识掌握并不太好,并且在完成多周期处理器时在设计环节花费的时间过少;如果仔细画出了状态机对于每类指令的,详细的,带控制信号变化的状态转移图,应该能减少大大因遗漏各类指令,各种控制信号而出现的问题。

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

● 思考题 1:ALUop 的编码有什么规律?资料表格中的 ALUop 编码是否有优化空间?。

ALUop 的编码规律主要在于功能分类与指令字段的直接映射。其低两位 op[1:0] 划定运算大类,如 10 为算术类、01/00 为逻辑类、11 为比较类;最高位(op[2])则作为功能修饰符,分别在算术中区分加减、在逻辑中区分基础与扩展、在比较中区分有符号与无符号。这种设计极大地方便了从 R-Type 指令的 func 字段直接提取控制信号,缩短了译码路径。

优化空间主要在于增强编码的正交性以简化控制逻辑。目前的编码方案下,sltu (011) 的最高位为 0,导致判定减法器触发条件(A-B)时,无法仅靠 op[2] 这一位完成,必须额外增加对 011 的特例识别逻辑。若将所有需要调用减法器的指令(减法及两类比较)统一编码为 op[2]=1,则 ALU 内部的 cin 和 B 端取反信号可直接由该位驱动,从而简化硬件电路并提升频率。

● 思考题 2:第二段状态机用 current_state 作为判断条件或赋值变量,尽量不使用 next_state?。

控制信号必须与当前阶段正在处理的数据保持同步,若使用 next_state,会导致控制信号在数据尚未准备好的前一个周期提前产生,从而破坏当前状态的逻辑运算。此外,current_state 来自触发器输出,在整个时钟周期内稳定,而 next_state 作为组合逻辑输出,在指令译码未完成前易产生毛刺和逻辑震荡,直接链接控制信号会大幅降低电路的可靠性并增加调试难度。

五、 实验时长

在课后,你花费了大约 18 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	2	单周期 1.5, 多周期 0.5
开发	7	单周期 4, 多周期 3
调试	9	单周期 5, 多周期 4

表 3: 实验时长